C L O U D C O M P U T I N G P R O J E C T B L U E P R I N T

ResumeRank: Serverless AI Resume Screening on AWS

An end-to-end, event-driven, fully serverless application that reads PDF resumes with Amazon Textract, extracts skills with Amazon Comprehend, scores candidates against a job description in DynamoDB and alerts recruiters through SNS – built in two days by two people on a single AWS Free Tier account

Domain · Cloud Computing · Serverless Architecture · Applied AI Services

Type of Project · Course Project (Team of Two · 2 Working Days · Cloud Budget Rs. 0)

Document · Project Blueprint, Version 1.0 (Two-Member Edition) · October 2026

8	2	2	~26 h	Rs. 0
AWS services	Lambda functions	DynamoDB tables	Total team effort	Expected AWS bill

All services used in this project fall within the AWS Free Tier at the usage levels described. The project is for educational purposes; it is not a production hiring system and does not make or recommend hiring decisions. Sample resumes used for testing are created by the team and contain no real personal data.

Contents

Executive Summary
1. How to Read This Document
2. The Problem, in Plain Terms
3. How Resume Screening Works Today
4. The Failure: The Manual-Screening Bottleneck
5. The Specific Gap: What a Serverless Design Changes
6. Problem Statement and Objectives
7. Scope of the Project
8. Solution Strategy
9. System Architecture and Data Flow
10. AWS Services: Role and Justification
11. Data Model (DynamoDB)
12. Scoring Methodology
13. The Free-Tier Compute Strategy
14. Security Design
15. Assumptions and Honest Scope
16. Technical Stack
17. Work Plan and Task Allocation
18. Hour-wise Division of Work (Day 1 and Day 2)
19. Integration and Coordination
20. Timeline and Milestones
21. Risk Assessment and Mitigation
22. Testing and Demonstration Protocol
23. Expected Outcomes and Deliverables
24. Viva Preparation
25. Glossary of Key Terms
References

Executive Summary

Every placement season, college placement cells and small companies receive hundreds of resumes per opening and screen them by eye. A recruiter spends on average a few seconds per resume; strong candidates are missed because keyword matching is done manually, and shortlists arrive days after applications close. Commercial Applicant Tracking Systems solve this for large firms at a monthly subscription that small teams cannot justify, and they require servers, logins and integration work that a placement cell cannot maintain.

Cloud-native, serverless architecture changes what a two-person team can build in two days. Amazon S3 can accept a file upload and emit an event; AWS Lambda can run code in response to that event without any server; Amazon Textract can read a PDF – including a scanned one – into plain text; Amazon Comprehend can pull skills, organisations and dates out of that text; DynamoDB can store the result with single-digit-millisecond reads; and Amazon SNS can email a recruiter within seconds. None of these components needs to be provisioned, patched or paid for while idle.

> **The project in one sentence**
>
> ResumeRank is a fully serverless, event-driven resume screening pipeline on AWS: a candidate uploads a PDF through a static web page; the upload triggers a Lambda function that OCRs the document with Textract, extracts skills and experience with Comprehend, scores the candidate 0–100 against the posted job's requirements, stores the result in DynamoDB, and emails the recruiter through SNS if the score crosses a threshold – with a live, ranked leaderboard served through API Gateway, full observability in CloudWatch, least-privilege IAM throughout, and a total cloud cost of Rs. 0 on the AWS Free Tier.

The work runs on two coordinated fronts. **Member A – Backend & AI Lead** owns the processing pipeline: the S3-triggered Lambda, the Textract and Comprehend integration, the scoring logic, DynamoDB writes and SNS alerts. **Member B – API, Frontend & Operations Lead** owns everything the user touches and everything that proves the system works: IAM roles and buckets, the API Gateway routes and their Lambda, the static web interface, CloudWatch dashboards, test data, the architecture diagram and the report. The two fronts meet at three fixed interfaces – the S3 key naming convention, the DynamoDB table schemas and the API contract – which are agreed in the first hour and never changed unilaterally. Deliverables are a working deployed system, a live demonstration, a GitHub repository, an

architecture diagram and a written report.

1. How to Read This Document

This blueprint is written to be read from start to finish. No prior experience with AWS is assumed; every service is introduced in plain language before it is used in the design.

> **Structure of this document**
>
> Sections 2–5 explain the problem and the cloud tools in plain language, one concept at a time.
>
> Sections 6–8 state the gap, the objectives, the scope and the solution strategy.
>
> Sections 9–16 describe the architecture, the services, the data model, the scoring method, the free-tier strategy, security, honest limits and the stack.
>
> Sections 17–22 cover the work plan, the hour-by-hour division between the two members, coordination, timeline, risks and testing.
>
> Sections 23–24 cover deliverables and viva preparation. Section 25 is a glossary; every technical term introduced in bold is defined there.

2. The Problem, in Plain Terms

2.1 What is resume screening?

Resume screening is the first filter in hiring: reading each applicant's resume and deciding whether they meet the minimum requirements of a job well enough to be interviewed. For a single opening a placement cell may receive 200–500 resumes. Screening produces a shortlist – typically the top 10–20 percent – and a reason for each decision.

> **Definition – automated screening**
>
> **Automated resume screening** means a program reads the resume, extracts structured facts (skills, years of experience, degrees), compares them against the job's requirements, and produces a score and a ranked list. A human still makes the hiring decision; the software only orders the pile and explains the order.

2.2 Why it matters: the placement-cell reality

Three properties of the setting shape the whole project. **First, volume is bursty**: nothing happens for months, then hundreds of resumes arrive in a week. A system that costs money while idle is wasteful; a system that cannot scale during the burst is useless. **Second, there is no IT team**: nobody will patch a server, renew a certificate or restart a crashed process. **Third, the budget is zero**: the tool must run on a free account.

2.3 What is serverless computing?

Serverless computing is a cloud model in which the provider runs your code in response to events and bills only for the milliseconds it executes. There are still servers, but you never see them: no instance to size, no operating system to patch, nothing running when there is no work. AWS Lambda is the compute part of this model; S3, DynamoDB, SNS and API Gateway are its storage, messaging and interface counterparts. Together they let a small team build a system that scales from zero to thousands of requests without changing a line of code.

3. How Resume Screening Works Today

Three approaches exist, and none fits the placement-cell setting.

Approach	How it works	Why it fails here
Manual review	Recruiters open each PDF and skim it	Slow, inconsistent between reviewers, no record of why a candidate was rejected
Keyword search in a folder	Search PDFs for required words	Misses synonyms ("JS" vs "JavaScript"), cannot read scanned PDFs, no scoring or ranking
Commercial ATS (Greenhouse, Lever, Zoho Recruit)	Hosted platform with parsing, ranking and workflow	Subscription cost, enterprise onboarding, far more features than needed, data leaves the institution

Self-hosted parser on a VM	Python script on an EC2 instance polling a folder	Instance runs 24×7 and bills 24×7; someone must maintain it; crashes go unnoticed

> **Definition – the managed-AI recipe**
>
> Modern cloud providers expose trained AI models as pay-per-call APIs: send a document, receive its text; send text, receive its entities. No model is trained, hosted or tuned by the caller. ResumeRank adopts this recipe – Textract for OCR, Comprehend for entity extraction – which is what makes a working AI pipeline achievable in two days with no machine-learning expertise.

4. The Failure: The Manual-Screening Bottleneck

The failure of current practice has three faces, and each maps to one part of the design.

- **The unstructured-input problem.** Resumes arrive in hundreds of layouts, and a meaningful fraction are scanned images with no selectable text. A simple text parser handles none of them reliably. The design answers with Textract, which performs OCR on any page layout.
- **The extraction problem.** Finding skills by exact keyword match misses abbreviations, synonyms and skills expressed in phrases ("built REST APIs in Flask"). The design answers with Comprehend key-phrase and entity detection plus a normalisation dictionary.
- **The latency and cost problem.** Shortlists built days after the deadline lose candidates to faster employers, while an always-on server costs money for the eleven idle months. The design answers with an event-driven pipeline: the resume is scored within seconds of upload, and nothing runs – or bills – between uploads.

> **Definition – event-driven architecture**
>
> An **event-driven architecture** is one in which components do nothing until an event occurs – a file lands in a bucket, a message arrives on a topic – and then react. ResumeRank's core is one such chain: S3 object-created event → Lambda → DynamoDB write → SNS publish. No component polls, waits or runs continuously.

5. The Specific Gap: What a Serverless Design Changes

Requirement	Traditional (VM-based) answer	Serverless answer in ResumeRank
Cost while idle	EC2 instance billed 24×7 (even t3.micro exits free tier after 750 h/month)	Rs. 0 – Lambda, S3, DynamoDB and SNS bill only per use
Scaling during placement week	Resize the instance or add a load balancer by hand	Lambda scales to 1,000 concurrent executions automatically
Reading scanned PDFs	Install Tesseract, poppler, manage dependencies	One Textract API call
Extracting skills	Write regexes or train an NER model	One Comprehend API call plus a 150-word dictionary
Operations	Patch OS, rotate logs, restart on crash	Nothing to patch; CloudWatch logs every invocation automatically
Security of uploads	Open an upload endpoint on the VM	Private bucket; browser uploads through a short-lived pre-signed URL

The gaps, stated compactly: **G1** – no free, maintenance-free screening tool exists for small placement cells; **G2** – keyword tools cannot read scanned resumes or handle synonyms; **G3** – no existing lightweight tool produces a transparent, explainable score with matched and missing skills per candidate; **G4** – course projects on AWS typically stop at a static site or a CRUD app and never combine event-driven compute with managed AI services.

6. Problem Statement and Objectives

6.1 Problem statement

> **Problem statement**
>
> **Current practice** – resumes are screened manually or by naive keyword search, producing slow, inconsistent and unexplained shortlists; the only alternatives are paid platforms or self-maintained servers.
>
> **Observed gap** – a small team with no budget and no operations staff has no tool that reads any resume, scores it objectively, explains the score and alerts the recruiter immediately.
>
> **Proposed remedy** – build a fully serverless, event-driven pipeline on AWS that OCRs uploaded resumes with Textract, extracts skills with Comprehend, computes a transparent 0–100 score against a job description stored in DynamoDB, serves a live ranked leaderboard through API Gateway, and notifies the recruiter through SNS – secured with pre-signed URLs and least-privilege IAM, observed through CloudWatch, and running entirely within the AWS Free Tier.

6.2 Objectives

- **O1** Build an event-driven serverless pipeline: S3 upload event → Lambda → Textract → Comprehend → DynamoDB → SNS, with no server provisioned at any stage.
- **O2** Apply managed AI services (Textract OCR, Comprehend NLP) without training or hosting a model.
- **O3** Design a NoSQL data model: two DynamoDB tables (Jobs, Candidates) with a composite key enabling "all candidates for job X, sorted by score" in one query.
- **O4** Expose a REST API through API Gateway with three routes and correct CORS configuration.
- **O5** Implement secure uploads: private S3 bucket, pre-signed PUT URLs valid for 5 minutes, IAM roles with only the permissions each Lambda needs.
- **O6** Deliver real-time notification: SNS email to the recruiter when a candidate scores ≥ 75.
- **O7** Make the system observable: CloudWatch logs for every invocation, a custom metric for scores, and a dashboard.
- **O8** Prove it: six test resumes with visibly different scores, a live demonstration, a GitHub repository, an architecture diagram and a written report – all within two working days and a Rs. 0 cloud bill.

7. Scope of the Project

The project is structured as one central claim – *a two-person team can build a production-shaped AI pipeline on AWS in two days for free* – supported on two fronts.

> **Front I – The processing pipeline (Member A)**
>
> **Question** – can an uploaded PDF be turned into a scored, explained candidate record automatically, within seconds, using only managed services?
>
> **Role** – this is the system's engine and the project's core demonstration of event-driven compute and managed AI.
>
> **Front II – The interface, security and operations layer (Member B)**
>
> **Question** – can the pipeline be exposed safely to a browser, observed by an operator and proved correct, without a server?
>
> **Role** – this is what makes the engine usable and demonstrable, and where the cloud-security and observability learning outcomes live.
>
> **Why the two fronts form one result**
>
> The pipeline without the interface is a script; the interface without the pipeline is a form. Together they answer the course question – what does a real cloud-native application look like, and why is it designed that way – rather than two unrelated exercises.

7.1 In and out of scope

In scope: single- and multi-page PDF resumes (text or scanned); one active job description at a time with multiple jobs supported in the data model; English-language resumes; a recruiter page and a candidate page; email alerts; CloudWatch logging and dashboard; six team-authored test resumes. **Out of scope for two days:** user authentication (Cognito); multiple recruiters; resume formats other than PDF; interview scheduling; any claim that the score predicts job performance; production hardening (WAF, custom domain, CI/CD). Each is named as future work in the report.

A scoping decision made on evidence

The first concept included Cognito login and a Step Functions state machine. A feasibility check found both added configuration time (3–4 hours) while contributing no additional learning on the core objectives – event-driven compute, managed AI, NoSQL design and secure uploads are untouched by their removal. Scope is therefore cut deliberately: pre-signed URLs provide upload security without login, and a single Lambda with sequential calls replaces Step Functions. Both are listed as the first two items of future work.

8. Solution Strategy

Building the system credibly in two days requires three things: a trusted foundation that cannot be redone later, a core pipeline that works end-to-end before it is polished, and a visible surface that proves it. The strategy supplies each in order.

Tool 1 – Foundation first: IAM, buckets, tables (Hour 1, both members)

What it does – creates the two S3 buckets, two DynamoDB tables, the SNS topic and the two IAM roles before any code is written, and agrees the three interfaces (S3 key format, table schema, API contract).

Why it is safe – every later task depends on these names; changing a table key on Day 2 would cost hours. Fixing them first removes the largest integration risk.

Tool 2 – The processing Lambda · primary contribution (Member A)

What it does – on an S3 object-created event, downloads nothing (Textract reads the object directly from S3), starts asynchronous text detection, polls for completion, sends the text to Comprehend, matches skills against the job's dictionary, computes the score, writes the Candidate record and publishes to SNS when the threshold is crossed.

Why it fits – every design choice maps to a constraint: S3 trigger → no polling; Textract async API → multi-page PDFs without any PDF library in Lambda; Comprehend → no model to train; sequential calls in one function → no orchestration service to configure.

Tool 3 – API, frontend and observability · the demonstrable surface (Member B)

What it does – API Gateway routes for creating a job, requesting an upload URL and listing ranked candidates; a static HTML/JS page on S3 that calls them; a CloudWatch dashboard that shows invocations, errors, duration and the custom score metric.

Why it is the standout – it turns the pipeline into something an examiner can click, and the dashboard turns "it works" into "here is the proof".

Fallbacks held in reserve

Textract async fails or is slow – fall back to the synchronous DetectDocumentText call and state "single-page PDF" as a limitation.

Comprehend returns poor phrases – the skill dictionary match on raw Textract text is already computed; Comprehend becomes an enrichment rather than a dependency.

CORS consumes the morning – Member A continues pipeline testing by uploading directly through the S3 console while Member B resolves it; the demo never depends on the browser upload alone.

Schedule slips – pre-declared cut order: CloudWatch dashboard → leaderboard auto-refresh → custom metric → architecture-diagram polish. The pipeline, the API, the upload page and the SNS alert are never cut.

9. System Architecture and Data Flow

The system has two paths that share one database: a **request path** (browser → API Gateway → Lambda → DynamoDB) and an **event path** (S3 → Lambda → Textract/Comprehend → DynamoDB → SNS). Every resume passes through the event path once; every page load uses the request path.



9.1 Stage-by-stage flow

Stage 1 – Job creation

The recruiter fills a form (title, required skills, minimum years, recruiter email). `POST /jobs` writes a Jobs item with a generated `jobId`; the required skills are stored as a lower-cased string set.

Stage 2 – Upload request

The candidate enters name and email and selects a PDF. `POST /upload-url` returns a pre-signed S3 PUT URL for the key `resumes/{jobId}/{candidateId}.pdf`, valid for 300 seconds, and simultaneously writes a Candidates item with status `PENDING`.

Stage 3 – Upload

The browser PUTs the file directly to S3 using the URL. The bucket remains private; no AWS credentials ever reach the browser.

Stage 4 – Trigger and OCR

S3 emits an `ObjectCreated:Put` event filtered to prefix `resumes/` and suffix `.pdf`. `process_resume` parses `jobId` and `candidateId` from the key, calls `StartDocumentTextDetection` with the S3 object location, and polls `GetDocumentTextDetection` every 3 seconds until `SUCCEEDED` (typically 8–20 s for a 2-page resume).

Stage 5 – Extraction

All LINE blocks are joined into one text string, truncated to 5,000 bytes per call; Comprehend `DetectKeyPhrases` and `DetectEntities` (types ORGANIZATION, DATE, TITLE) are called on each chunk.

Stage 6 – Scoring and persistence

Skills are matched (Section 12), experience years estimated, education detected, score computed; the Candidates item is updated with score, matched skills, missing skills, status `SCORED` and the first 300 characters of extracted text as a preview. A custom CloudWatch metric `ResumeRank/CandidateScore` is emitted.

Stage 7 – Notification and display

If score ≥ 75, the function publishes to the SNS topic; the recruiter receives an email naming the candidate, score and matched skills. The recruiter page polls `GET /candidates?jobId=` every 5 seconds and renders the ranked table.

10. AWS Services: Role and Justification

Service	Role in ResumeRank	Why this service	Free-tier allowance
Amazon S3	Hosts the static website; stores uploaded resumes privately; emits the trigger event	Durable object storage with native event notifications and pre-signed URLs	5 GB storage, 20,000 GET, 2,000 PUT per month (12 months)
AWS Lambda	Runs both the API handler and the resume processor	No server; scales to zero; native S3 and API Gateway integration	1 million requests and 400,000 GB-seconds per month (always free)

Amazon API Gateway (HTTP API)	Exposes three REST routes to the browser with CORS	Managed, pay-per-call HTTP front door for Lambda; HTTP API is cheaper and simpler than REST API	1 million calls per month (12 months)
Amazon Textract	OCR: PDF → text lines	Reads scanned and text PDFs with no library inside Lambda; async API handles multi-page	1,000 pages per month for DetectDocumentText (3 months)
Amazon Comprehend	NLP: key phrases, organisations, dates, titles	Pre-trained entity and phrase extraction; no model to train	50,000 units (5 million characters) per month (12 months)
Amazon DynamoDB	Stores Jobs and Candidates; serves the leaderboard query	Serverless NoSQL; single-digit ms reads; composite key gives sorted-by-job queries	25 GB storage, 25 RCU and 25 WCU (always free)
Amazon SNS	Emails the recruiter on high-scoring candidates	One-line publish; email subscription needs no SMTP setup	1,000 email deliveries per month (always free)
Amazon CloudWatch	Logs every invocation; custom score metric; dashboard; billing alarm	Automatic for Lambda; the project's observability evidence	10 custom metrics, 5 GB logs, 10 alarms, 3 dashboards (always free)
AWS IAM	Two execution roles with least-privilege policies	Mandatory for Lambda; the security learning outcome	Free

Free-tier figures are the published allowances at the time of writing. AWS has moved newly created accounts to a credit-based free plan; on either model this project's expected usage – roughly 30 Textract pages, 60 Comprehend calls, 500 Lambda invocations and under 10 MB of storage across both days – is far below any limit. A billing alarm at USD 1 is created in Hour 1 regardless.

11. Data Model (DynamoDB)

Two tables. The Candidates table uses a composite primary key so that "all candidates for one job" is a single `Query` on the partition key, and a Global Secondary Index sorts them by score without scanning.

Table: Jobs	Type	Description
jobId (PK)	String	UUID generated by api_handler
title	String	Job title, e.g. "Backend Developer Intern"
requiredSkills	String set	Lower-cased canonical skill names, e.g. {"python","aws","sql","docker"}
minYears	Number	Minimum experience required (0 for fresher roles)
recruiterEmail	String	SNS subscription target
createdAt	String	ISO-8601 timestamp

Table: Candidates	Type	Description
jobId (PK)	String	Partition key – groups all candidates of one job
candidateId (SK)	String	Sort key – UUID per upload
name, email	String	Entered on the upload form
status	String	PENDING → SCORED | FAILED
score	Number	0–100 composite score (GSI sort key)
skillScore, expScore, eduScore	Number	Component scores for explainability
matchedSkills, missingSkills	List	Shown on the leaderboard
yearsExperience	Number	Estimated from DATE entities and regex
textPreview	String	First 300 characters of OCR output
s3Key, processedAt	String	Traceability

GSI: `ScoreIndex` with partition key `jobId` and sort key `score`, projection ALL, queried with `ScanIndexForward=False` to return the leaderboard already sorted descending. Both tables use on-demand capacity, which stays inside the always-free allowance at this volume.

12. Scoring Methodology

The score is deliberately a transparent weighted heuristic rather than a learned model: every number on the leaderboard can be explained to a candidate or an examiner in one sentence. Three components sum to 100.

```
skill_score = (|matched_skills| / |required_skills|) × 60
exp_score   = min(years_found / max(min_years, 1), 1.0) × 20     (fresher roles: 20 if any internship/project dates found)
edu_score   = 20 if any degree keyword found (b.tech, b.e., bachelor, m.tech, mca, bsc, msc, phd) else 10
total       = round(skill_score + exp_score + edu_score)
```

12.1 Skill matching

A dictionary of ~150 canonical skills with aliases (e.g. `"javascript": ["js", "java script", "ecmascript"]`) is bundled with the Lambda. Matching runs in two passes: an exact alias search over the lower-cased Textract text, then a search over Comprehend key phrases, which catches skills embedded in phrases ("deployed on Amazon Web Services"). The union is intersected with the job's `requiredSkills` to produce `matchedSkills`; the set difference produces `missingSkills`.

12.2 Experience estimation

Two signals are combined and the larger is taken: (a) explicit patterns via regex – `(\d+)\+?\s*(years?|yrs?)`; (b) date ranges from Comprehend DATE entities – year pairs such as "2021 – 2023" or "Jun 2022 – Present" are summed. The result is capped at 15 years.

> **An honest point about the score**
>
> The score ranks resumes by stated fit to a job description; it does not measure ability, and the report says so explicitly. Resumes with unusual layouts may OCR imperfectly; a candidate who writes "Amazon Web Services" scores the same as one who writes "AWS" only because the alias dictionary says so. These are stated limitations, and the component scores are stored precisely so that a human can see why a number is what it is.

13. The Free-Tier Compute Strategy

The project is designed so that every component stays inside the free allowance by construction, not by luck.

> **The strategy that makes Rs. 0 achievable**
>
> **Nothing runs continuously.** There is no EC2 instance, no NAT gateway, no RDS database, no load balancer – the four services most likely to produce a surprise bill on a student account. Every component is invoked per request or per event.
>
> **AI calls are counted, not unlimited.** Thirty test uploads of two-page resumes use ~60 Textract pages against a 1,000-page monthly allowance, and ~60 Comprehend calls of under 5,000 characters each against a 5-million-character allowance.
>
> **Lambda is sized small.** 256 MB memory, 120-second timeout for the processor (polling Textract), 10-second timeout for the API handler. Total compute for the whole project is under 1 GB-second per upload.
>
> **Guard-rails are built in Hour 1.** A CloudWatch billing alarm at USD 1 emails both members; the AWS Budgets free-tier usage alert is enabled; S3 lifecycle rule deletes uploaded resumes after 7 days.
>
> **Tear-down is scheduled.** After the final demonstration, all resources are deleted in a documented order (Section 22) so the account returns to zero usage.

Resource	Expected 2-day usage	Free allowance
Lambda invocations	~500	1,000,000 / month
Textract pages	~60	1,000 / month
Comprehend characters	~250,000	5,000,000 / month
DynamoDB storage / requests	< 1 MB / ~2,000	25 GB / 25 RCU+WCU continuous
S3 storage / requests	~6 MB / ~500	5 GB / 22,000
API Gateway calls	~1,500 (leaderboard polling)	1,000,000 / month
SNS emails	~15	1,000 / month

14. Security Design

- **Private resume bucket.** Block Public Access ON; no bucket policy grants public read. Resumes are PII-adjacent and never exposed.
- **Pre-signed URLs.** The API handler signs a PUT for one specific key, valid 300 seconds, content-type restricted to `application/pdf`. The browser never holds AWS credentials.
- **Least-privilege IAM.** `role-api`: `dynamodb:PutItem/GetItem/Query` on the two tables, `s3:PutObject` on `resumes/*` only. `role-processor`: `s3:GetObject` on `resumes/*`, `textract:StartDocumentTextDetection/GetDocumentTextDetection`, `comprehend:DetectKeyPhrases/DetectEntities`, `dynamodb:UpdateItem/GetItem`, `sns:Publish` on the one topic, `cloudwatch:PutMetricData`, plus the managed `AWSLambdaBasicExecutionRole` for logs. No wildcard resources except where the API has none.
- **CORS.** API Gateway allows only the website bucket's origin; the resume bucket's CORS allows PUT from that origin only.
- **Input validation.** The API handler rejects missing fields, non-PDF content types and files over 5 MB (enforced through the pre-signed URL's content-length condition).
- **Data retention.** S3 lifecycle rule expires resumes after 7 days; the report notes that a production system would add encryption-at-rest with a customer-managed KMS key and Cognito authentication.

15. Assumptions and Honest Scope

> **What is and is not claimed**
>
> **Prototype, not product** – the project delivers a working, deployed pipeline framed as a course demonstration, not a production hiring system.
>
> **English, PDF, single job focus** – the demo uses English resumes in PDF; the data model supports multiple jobs but the UI shows one at a time.
>
> **Heuristic scoring** – the score is a transparent rule, not a trained model, and makes no claim about candidate quality.
>
> **No authentication** – anyone with the URL can post a job or upload a resume; this is the first named item of future work.
>
> **Rs. 0 means Rs. 0** – the feasibility claim counts free-tier usage only; the billing alarm and tear-down protocol are part of the deliverable.
>
> **Fairness** – automated screening can encode bias (e.g. penalising career gaps). The report includes a short ethics section stating that the tool orders a pile for human review and must never auto-reject.

16. Technical Stack

Component	Description
Region	ap-south-1 (Mumbai) – Textract and Comprehend both available; lowest latency from India
Runtime	Python 3.12 on Lambda; boto3 (bundled in the Lambda runtime, no packaging needed)
Lambda functions	`api_handler` (256 MB, 10 s) · `process_resume` (256 MB, 120 s)
API	API Gateway HTTP API, three routes, Lambda proxy integration, CORS enabled
Frontend	Two static HTML pages with vanilla JavaScript and fetch(); no framework, no build step; hosted on S3 static website
Storage	S3 (website bucket, resumes bucket) · DynamoDB on-demand (Jobs, Candidates + ScoreIndex GSI)
AI services	Textract (async StartDocumentTextDetection) · Comprehend (DetectKeyPhrases, DetectEntities)
Messaging / monitoring	SNS email topic · CloudWatch Logs, custom metric, dashboard, billing alarm
Tooling	AWS Console for all resources (fastest for a 2-day scope); AWS CLI for Lambda zip deploys; Git + GitHub; draw.io for the architecture diagram; Postman or curl for API tests
Test data	Six team-authored resumes: 3 strong, 2 partial, 1 scanned image-PDF – no real personal data

17. Work Plan and Task Allocation

The project is executed by a **two-member team over two working days (~13 hours each)**. Responsibility is divided along the natural seam of the system. **Member A – Backend & AI Lead** owns everything that happens after a file lands in S3: the processing Lambda, Textract and Comprehend integration, the skill dictionary, scoring, DynamoDB writes, SNS publishing and the custom metric. **Member B – API, Frontend & Operations Lead** owns everything before the file lands and everything that proves the system works: IAM roles, buckets, tables, API Gateway, the API Lambda, the web pages, CORS, CloudWatch dashboard, test resumes, the architecture diagram and the report assembly. Both share Hour 1 setup, all integration tests and the viva.

> **Division principle**
>
> **Member A** answers: does the pipeline turn a PDF into a correct score, automatically?
>
> **Member B** answers: can a user reach it safely, and can we prove it works?
>
> **Shared** – Hour 1 foundation, end-of-day integration tests, demo rehearsal, report review, viva.
>
> The interfaces between the two are the S3 key convention, the DynamoDB schemas and the API contract (Section 19): agreed in Hour 1, written in `INTERFACES.md` in the repo, and never changed unilaterally.

Workstream 1 – Processing Pipeline (Member A)

Task	Deliverable
T1.1 Processor skeleton	`process_resume` Lambda parsing the S3 event key into jobId/candidateId; logs to CloudWatch; S3 trigger attached
T1.2 Textract integration	Async start + poll loop; full text assembled from LINE blocks; tested on a text PDF and a scanned PDF
T1.3 Comprehend integration	Chunked key-phrase and entity calls; organisations, dates, titles collected
T1.4 Skill dictionary and scoring	`skills.json` (~150 skills with aliases); `scoring.py` implementing Section 12; unit-tested with three sample texts
T1.5 Persistence and alerts	DynamoDB UpdateItem with all fields; SNS publish on score ≥ 75; custom metric emitted
T1.6 Error handling	try/except around each AWS call; status FAILED written with reason; Lambda retry disabled to avoid duplicate scoring
T1.7 Technical sections of report	Architecture, pipeline, scoring, AI-services and security sections

Workstream 2 – Foundation, API and Frontend (Member B)

Task	Deliverable
T2.1 Foundation	Two S3 buckets, two DynamoDB tables with GSI, SNS topic + confirmed subscription, two IAM roles, billing alarm, `INTERFACES.md`
T2.2 API Lambda	`api_handler` implementing POST /jobs, POST /upload-url (pre-signed URL + PENDING item), GET /candidates (GSI query)
T2.3 API Gateway	HTTP API with three routes, Lambda proxy integration, CORS configured; tested with curl
T2.4 Frontend	`recruiter.html` (post job + ranked table with 5 s refresh) and `upload.html` (form + pre-signed PUT); S3 website hosting enabled
T2.5 Test data	Six resumes authored with deliberately varied skills/experience; expected scores recorded

Workstream 3 – Operations, Evidence and Dissemination (Member B, with Member A for integration)

Task	Deliverable
T3.1 Observability	CloudWatch dashboard: invocations, errors, duration, CandidateScore metric; screenshots for report
T3.2 Architecture diagram	draw.io diagram using official AWS icons, exported as PNG
T3.3 Integration testing	End-to-end run of all six resumes; results table (expected vs actual score)
T3.4 Repository	GitHub repo: `lambda/`, `frontend/`, `iam/` (policy JSON), `docs/`, README with deployment steps
T3.5 Report and demo	Report assembled from both members' sections; demo script; tear-down checklist executed after viva

18. Hour-wise Division of Work

Both members work in parallel through every block; the table states what each owns per block and the joint checkpoint at block end. The sequencing rule: an end-to-end path exists by the end of Day 1, and every Day 2 block adds rather than gates.

18.1 Day 1 – Foundation and the end-to-end path (~7 hours)

Block	Member A – Backend & AI	Member B – API, Frontend & Ops	Joint checkpoint
Hr 1	Create AWS account access for both (IAM user each, MFA on root); agree the three interfaces and write `INTERFACES.md`; create Git repo	Billing alarm (USD 1) and Free Tier usage alert; create `resumerank-site` and `resumerank-resumes` buckets (Block Public Access on both for now); create Jobs and Candidates tables + ScoreIndex GSI; SNS topic with both emails subscribed and confirmed	**C1** – Foundation exists; interfaces frozen; both can log in
Hr 2	Create `role-processor` with the Section 14 policy; create `process_resume` Lambda (Python 3.12, 256 MB, 120 s); attach S3 trigger on `resumes/` + `.pdf`; upload a PDF through the console and confirm the event arrives in CloudWatch Logs	Create `role-api`; create `api_handler` Lambda; implement POST /jobs (write Jobs item) and test from the Lambda console with a test event	Both Lambdas invoke successfully
Hr 3–4	Textract integration: StartDocumentTextDetection → poll GetDocumentTextDetection → join LINE blocks; log the first 500 characters; test on a 1-page text PDF, then a 2-page PDF, then a scanned PDF	Implement POST /upload-url (generate_presigned_url with ContentType condition, write PENDING Candidates item) and GET /candidates (Query on ScoreIndex, ScanIndexForward=False); create HTTP API with three routes + Lambda proxy; test all three with curl	**C2** – Text comes out of any PDF; API answers all three routes
Hr 5	Comprehend integration: chunk text at 5,000 bytes; DetectKeyPhrases + DetectEntities; collect phrases, ORGANIZATION, DATE, TITLE; log counts	Author the six test resumes (3 strong, 2 partial, 1 scanned image-PDF made by printing to image and re-saving as PDF); record expected scores in `docs/test_matrix.md`	Entities visible in logs; test set ready
Hr 6	Build `skills.json` and `scoring.py`; implement Section 12 formulas; unit-test locally with three pasted texts	Write `upload.html`: form → POST /upload-url → PUT to S3; enable static website hosting on the site bucket (public read on that bucket only); first browser upload attempt – expect CORS errors and begin fixing	Score computed correctly on sample text
Hr 7	Wire scoring into the Lambda: UpdateItem with score, components, matched/missing skills, status SCORED; SNS publish on ≥ 75; emit CandidateScore metric	Resolve CORS on API Gateway and on the resumes bucket; confirm browser upload lands in S3 and triggers Member A's function	**C3 – END OF DAY 1:** upload from browser → email arrives → item in DynamoDB. The end-to-end path exists.

18.2 Day 2 – Interface, evidence and dissemination (~6 hours)

Block	Member A – Backend & AI	Member B – API, Frontend & Ops	Joint checkpoint
Hr 1	Error handling: try/except per AWS call, FAILED status with reason, set Lambda async retry to 0; run all six test resumes and compare to expected scores; tune aliases where a legitimate skill was missed	Write `recruiter.html`: post-job form, ranked table (rank, name, score, matched, missing, status), 5-second auto-refresh with fetch(); colour-code score bands	Six resumes scored; recruiter page renders
Hr 2	Experience-estimation refinement (date-range summing from Comprehend DATE entities); cap and sanity checks; re-run tests; freeze the processor – no logic changes after this	CloudWatch dashboard: Lambda invocations, errors, duration (both functions), CandidateScore metric; take screenshots; verify the billing alarm by viewing its state	**C4** – Processor frozen; dashboard shows the Day-2 runs
Hr 3	Write the technical report sections: architecture narrative, pipeline stages, scoring methodology, AI-service justification, security design; export IAM policy JSON into `iam/`	Draw the architecture diagram in draw.io with official AWS icons; export PNG; assemble repo structure and README (deploy steps, screenshots, limitations)	Diagram and repo skeleton done
Hr 4	Cross-verify Member B's work: run through the README deploy steps as a reader; test the recruiter page with a fresh job	Cross-verify Member A's work: upload two new resumes and check the scores against the formula by hand; confirm the SNS email contents	**C5** – Every headline claim verified by the member who did not build it
Hr 5	Report: limitations, ethics and fairness note, future scope (Cognito, Step Functions, Bedrock-based scoring, KMS, CI/CD), cost analysis table (Section 13)	Report: problem statement, objectives, services table, testing results table (expected vs actual), screenshots, deliverables; merge both halves; export PDF	Report complete

Hr 6	Demo rehearsal: drive the demo while B narrates; prepare technical viva answers (Section 24)	Demo rehearsal: narrate while A drives; prepare the operations/security viva answers; final Git push and tag `v1.0`	**C6 – END OF DAY 2:** rehearsed demo, report PDF, tagged repo, diagram

> **Load-balancing rule**
>
> If either member's track slips more than one hour, the other absorbs the most parallelisable task from the slipping track. Test-resume authoring, dashboard creation, diagram drawing and report sections are all transferable; the Textract/Comprehend integration and the API Lambda are not, because each depends on context the other member has not built. The pre-declared cut order of Section 8 applies before any deliverable is descoped; the pipeline, the API, the upload page and the SNS alert are never cut regardless of whose track slipped.

19. Integration and Coordination

Because the two tracks produce one system, coordination is engineered into the artefacts rather than relied on as goodwill.

> **Shared interfaces (written in INTERFACES.md in Hour 1)**
>
> **S3 key convention** – `resumes/{jobId}/{candidateId}.pdf`. Member B's API writes keys in this form; Member A's processor parses them. Neither changes it.
>
> **DynamoDB schemas** – exactly as Section 11. Member B creates the tables and writes PENDING items; Member A updates them to SCORED. Attribute names are copied from the document, not retyped.
>
> **API contract** – request and response JSON for the three routes, with one example each. Member B implements; both the frontend and the test scripts consume.
>
> **Checkpoints**
>
> **Six checkpoints C1–C6** (Section 18) each name one observable fact; a checkpoint is passed only when both members have seen the fact, not when one reports it.
>
> **Single shared AWS account, separate IAM users** – both members see the same resources; CloudTrail (on by default) records who changed what.
>
> **Quality control**
>
> **Cross-verification** (Day 2, Hr 4) – every headline claim is checked by the member who did not build it, before the report is written.
>
> **No logic changes after the Day-2 Hr-2 freeze** – later hours measure and document the frozen system; a bug found after the freeze is fixed only if it breaks the demo, and the fix is re-verified by the other member.
>
> **Joint authoring** – both members write the report; neither section is submitted without the other's read-through.

20. Timeline and Milestones

Phase	When	Focus and key artefacts	Checkpoint / gate
Phase 1	Day 1, Hr 1–2	Foundation: account, IAM, buckets, tables, SNS, billing alarm, interfaces, both Lambda shells with triggers	**C1** foundation exists · both Lambdas invoke
Phase 2	Day 1, Hr 3–5	Core integrations: Textract text extraction on three PDF types; Comprehend entities; API routes live on API Gateway; test resumes authored	**C2** text from any PDF · API answers all routes
Phase 3	Day 1, Hr 6–7	Scoring wired in; DynamoDB writes; SNS alert; upload page; CORS resolved	**C3** END OF DAY 1 – end-to-end path exists
Phase 4	Day 2, Hr 1–2	Hardening and evidence: error handling, six-resume test run, experience refinement, recruiter leaderboard, CloudWatch dashboard, processor freeze	**C4** processor frozen · dashboard populated
Phase 5	Day 2, Hr 3–6	Dissemination: report, diagram, repository, cross-verification, demo rehearsal, tag v1.0	**C5** cross-verified · **C6** END OF DAY 2 – submission-ready

21. Risk Assessment and Mitigation

Risk	Mitigation
CORS errors consume Day 1 afternoon (most common failure)	Member B starts CORS in Hr 6, not Hr 7; Member A keeps testing via console uploads so the pipeline track never blocks; a known-good CORS configuration is written into INTERFACES.md in advance (allow origin = website URL, methods GET/POST/PUT/OPTIONS, headers Content-Type)
Textract async job exceeds Lambda timeout	120-second timeout with 3-second polling handles resumes up to ~10 pages; the API rejects files over 5 MB; fallback to synchronous DetectDocumentText with a single-page limitation
Lambda retries cause duplicate scoring / duplicate emails	Asynchronous invocation retry set to 0; processor is idempotent on candidateId (UpdateItem, not PutItem)
Scanned resume OCR is poor	Accept; show the text preview on the leaderboard so the recruiter sees why; document as limitation. Textract handles typical 300-dpi scans well
Free-tier limit hit or unexpected charge	Usage is <10 % of every allowance; USD 1 billing alarm emails both members; no always-on service exists in the design; tear-down after viva
SNS subscription not confirmed	Both members confirm the subscription email in Hr 1 and verify topic shows "Confirmed"
IAM permission errors slow Day 1	Policies are written from Section 14 verbatim; CloudWatch log line names the missing action – add it, do not broaden to wildcard
One member unavailable for a block	Load-balancing rule (Section 18); all code is in Git after every block, never only on one laptop
Demo-day network or console failure	Report carries screenshots of every stage; a recorded screen video of the full flow is made during Day 2 Hr 6 rehearsal as a backup

22. Testing and Demonstration Protocol

22.1 Test matrix

#	Resume design	Purpose	Expected score band
R1	All required skills, 3 years, B.Tech, text PDF	Upper bound; SNS alert must fire	90–100
R2	All skills written as aliases ("JS", "Amazon Web Services"), 2 years	Alias dictionary	85–95
R3	Same as R1 but scanned image-PDF	OCR path	85–100
R4	Half the skills, 1 year, no degree keyword	Mid band; no alert	40–55
R5	No required skills, 5 years, M.Tech	Skill weight dominates	30–40
R6	Two-page resume with skills on page 2	Multi-page Textract	Matches R1 ± 5

22.2 Demonstration script (6 minutes)

1. Show the architecture diagram; state the one-sentence claim. (45 s)
2. Recruiter page: post a job "Cloud Engineer Intern" with skills python, aws, docker, linux, sql. (45 s)
3. Upload page: upload R1. Switch to the recruiter page; within ~20 seconds the candidate appears with score and matched skills. Show the SNS email arriving on a phone. (90 s)
4. Upload R4; show it ranks below R1 with missing skills listed. (45 s)
5. Upload R3 (scanned); show the text preview proving OCR worked. (45 s)
6. CloudWatch dashboard: invocations, durations, score metric; the DynamoDB item; the IAM policy; the private bucket setting. (60 s)
7. State limitations and future work. (30 s)

22.3 Tear-down checklist (after viva)

Delete in order: S3 trigger → both Lambdas → API Gateway → empty and delete both buckets → both DynamoDB tables → SNS topic → CloudWatch dashboard and alarm → both IAM roles and policies. Confirm the Billing console shows zero forecast.

23. Expected Outcomes and Deliverables

- **Deployed system** – a working serverless pipeline on AWS: static site, HTTP API, two Lambdas, Textract, Comprehend, DynamoDB, SNS and CloudWatch, live at demonstration time.

- **Live demonstration** – the six-minute script of Section 22, including a scanned resume and a real-time email alert.
- **GitHub repository** – Lambda source, frontend, IAM policy JSON, skill dictionary, test matrix, README with step-by-step deployment and tear-down.
- **Architecture diagram** – draw.io source and PNG with official AWS icons.
- **Project report** – 12–18 pages: abstract, problem statement, objectives, architecture, service justification, data model, scoring methodology, security, testing results, cost analysis, limitations and ethics, future scope, conclusion; screenshots of every stage.
- **Cost evidence** – Billing console screenshot showing Rs. 0 and the alarm configuration.
- **Backup video** – a screen recording of the full flow in case of demo-day failure.

#	Deliverable	Format	Weight	Evaluation criteria
D1	Working deployed pipeline	Live AWS	30 %	End-to-end flow works; event-driven; no server
D2	Live demonstration	6-min demo	15 %	Clear, rehearsed, handles the scanned case and the alert
D3	Code repository	GitHub	15 %	Readable, documented, reproducible from README
D4	Architecture and security design	Diagram + IAM	15 %	Correct service choices justified; least privilege; private bucket
D5	Report	PDF	15 %	Completeness, cost analysis, honest limitations
D6	Viva	Oral	10 %	Both members can explain every component

24. Viva Preparation

Each member prepares the questions in their own track and can answer the other's at a summary level.

Likely question	Prepared answer (summary)
Why serverless instead of EC2?	Bursty workload, zero idle cost, automatic scaling, no OS to patch; EC2 would bill 24×7 and exceed free tier after 750 h
What triggers the processing Lambda?	S3 ObjectCreated:Put event notification filtered on prefix resumes/ and suffix .pdf – push, not polling
Why the async Textract API?	Synchronous DetectDocumentText accepts single-page documents only; async handles multi-page PDFs directly from S3 with no PDF library in Lambda
How does the browser upload without AWS credentials?	api_handler signs a PUT URL with its own role for one key, 300 s, PDF content-type; the bucket stays private
Why two tables and a GSI?	Jobs and Candidates have different access patterns; GSI on (jobId, score) returns the ranked list in one sorted Query instead of a Scan
What is least privilege here?	Each role lists only the actions and resource ARNs its function uses; no "*" resources on DynamoDB, S3 or SNS
What happens if Textract fails?	Exception caught; Candidates item set to FAILED with reason; retry disabled so no duplicate; visible on the leaderboard and in CloudWatch
Is the score fair?	It is a transparent rule on stated skills, experience and education; it orders a pile for human review and must not auto-reject; bias risks are stated in the report
How much does it cost?	Rs. 0 – usage under 10 % of every free allowance; billing alarm proves it; no always-on resource
How would you productionise it?	Cognito auth, Step Functions for the pipeline, KMS encryption, WAF on API Gateway, infrastructure as code (SAM/CDK), CI/CD, Bedrock-based semantic scoring

25. Glossary of Key Terms

Term	Meaning
Serverless computing	Running code in response to events with no server to provision; billed per execution
Event-driven architecture	Components act only when an event (file created, message published) occurs

AWS Lambda	AWS's function-as-a-service; runs a function per event, scales automatically
Amazon S3	Object storage; here hosts the site, stores resumes and emits upload events
Pre-signed URL	A time-limited URL carrying a signature that grants one specific S3 operation without sharing credentials
API Gateway (HTTP API)	Managed front door that maps HTTP routes to Lambda functions
CORS	Browser rule allowing a page from one origin to call an API on another; must be configured on API Gateway and S3
Amazon Textract	Managed OCR and document-analysis service; returns text blocks from PDFs and images
Amazon Comprehend	Managed NLP service; returns key phrases and named entities from text
DynamoDB	Serverless NoSQL key-value database with partition and sort keys
Global Secondary Index (GSI)	An alternative key on a DynamoDB table enabling a different query pattern (here: sort by score)
Amazon SNS	Publish/subscribe messaging; here delivers email alerts
CloudWatch	Logs, metrics, dashboards and alarms for AWS resources
IAM role / least privilege	An identity a Lambda assumes; least privilege means granting only the actions and resources it needs
Idempotent	An operation that produces the same result if repeated; here protects against duplicate scoring
Free Tier	AWS's no-charge usage allowances (some for 12 months, some permanent, some credit-based on newer accounts)
Skill dictionary	A JSON map of canonical skill names to their aliases used for matching
Composite score	The weighted 0–100 sum of skill, experience and education components

References

[1] Amazon Web Services, "AWS Lambda Developer Guide," docs.aws.amazon.com/lambda, accessed October 2026.

[2] Amazon Web Services, "Amazon Textract Developer Guide – Detecting Text (asynchronous operations)," docs.aws.amazon.com/textract, accessed October 2026.

[3] Amazon Web Services, "Amazon Comprehend Developer Guide – DetectKeyPhrases and DetectEntities," docs.aws.amazon.com/comprehend, accessed October 2026.

[4] Amazon Web Services, "Amazon DynamoDB Developer Guide – Global Secondary Indexes," docs.aws.amazon.com/dynamodb, accessed October 2026.

[5] Amazon Web Services, "Amazon S3 User Guide – Using presigned URLs; Event notifications," docs.aws.amazon.com/s3, accessed October 2026.

[6] Amazon Web Services, "Amazon API Gateway – Configuring CORS for HTTP APIs," docs.aws.amazon.com/apigateway, accessed October 2026.

[7] Amazon Web Services, "AWS Free Tier," aws.amazon.com/free, accessed October 2026.

[8] Amazon Web Services, "AWS Well-Architected Framework – Serverless Applications Lens," 2024.

[9] P. Sbarski and S. Kroonenburg, *Serverless Architectures on AWS*, 2nd ed., Manning, 2022.

[10] M. Raghavan, S. Barocas, J. Kleinberg and K. Levy, "Mitigating Bias in Algorithmic Hiring: Evaluating Claims and Practices," in *Proc. ACM FAT**, pp. 469–481, 2020.

Note: references 1–7 are vendor documentation for the services used; 8–9 are architectural references; 10 grounds the ethics section.